ЛАБОРАТОРНА РОБОТА №6

СИСТЕМНЕ ПРОГРАМУВАННЯ

ФАЙЛОВЕ ВВЕДЕННЯ-ВИВЕДЕННЯ

Заяць Діани Юріївни

Хід виконання роботи:

6.1. Побудова ієрархічної структури директорії

Для виконання першого завдання було реалізовано механізм побудови повної ієрархічної структури директорії. Після запуску програми користувач може вибрати кореневу папку за допомогою кнопки огляду, після чого її шлях підставляється у відповідне текстове поле. Після натискання кнопки завантаження програма перевіряє коректність шляху, створює об’єкт типу DirectoryInfo та починає рекурсивне сканування директорії.

Основним методом, який запускає побудову дерева, є LoadDirectoryTree. У цьому методі виконується очищення попереднього вмісту елементів інтерфейсу, створення кореневого вузла дерева, перевірка існування каталогу, а також виклик рекурсивного методу для подальшого обходу вкладених директорій. Для того щоб процес оновлення дерева не викликав зайвого мерехтіння інтерфейсу, використовуються методи BeginUpdate та EndUpdate.

Безпосереднє рекурсивне додавання вузлів до TreeView реалізовано в методі AddDirectoryNodes. У ньому спочатку здійснюється перебір усіх вкладених папок поточного каталогу, після чого для кожної папки створюється новий вузол дерева та виконується повторний виклик цього самого методу для ще глибшого обходу структури. Після обробки всіх підкаталогів виконується додавання файлів поточного каталогу. Таким чином, у дереві формується повна вкладена структура, що містить як папки, так і файли.

Для збереження інформації про тип вузла і його повний шлях використовуються допоміжні структури TreeNodeTag, завдяки яким програма може коректно визначати, чи відповідає вибраний вузол файлу або папці, і виконувати відповідну логіку. Такий підхід спрощує подальшу навігацію по структурі та підвищує надійність програми.

У результаті виконання цього завдання було створено зручний інструмент для перегляду ієрархії файлової системи в графічному вигляді, що повністю відповідає вимогам методички щодо відображення структури директорії.

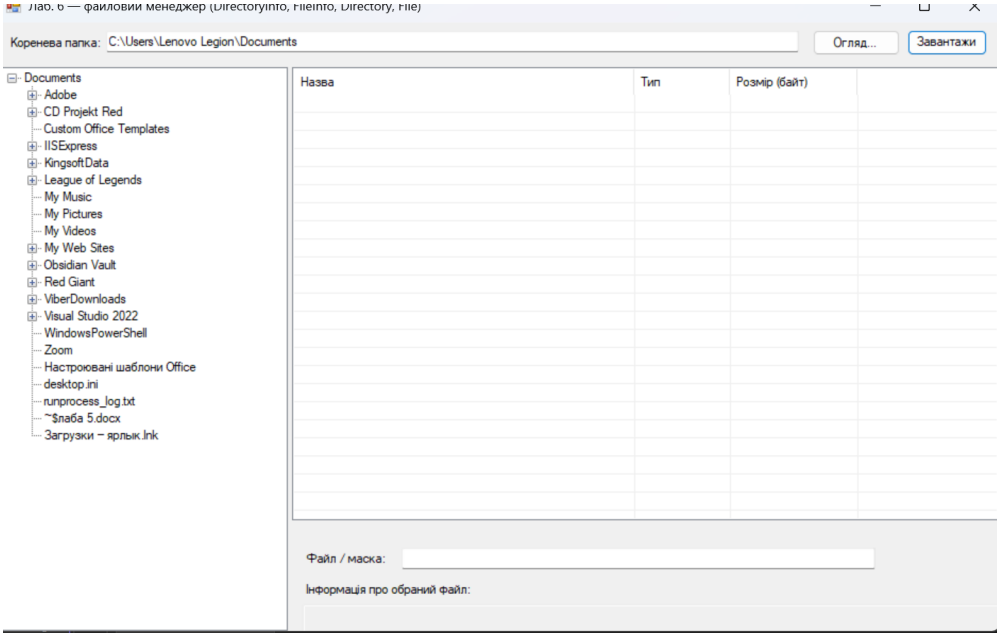

Рис. 6.1.1 – Головне вікно програми після вибору кореневої директорії з деревом

```
private void LoadDirectoryTree(string path)
{
    treeViewMain.BeginUpdate();
    try
    {
        treeViewMain.Nodes.Clear();
        listViewContents.Items.Clear();
        textBoxFileInfo.Clear();

        DirectoryInfo rootDir;
        try
        {
            //об'єктна модель каталогу: шлях передаємо в конструктор
            rootDir = new DirectoryInfo(path);
        }
        catch (Exception ex)
        {
            ShowError("Некоректний шлях до каталогу.", ex);
            return;
        }

        if (!rootDir.Exists)
        {
            MessageBox.Show(this, "Каталог не існує: " + path, "Помилка", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        var rootNode = new TreeNode(rootDir.Name)
        {
            Tag = new TreeNodeTag { IsDirectory = true, FullPath = rootDir.FullName }
        };
        treeViewMain.Nodes.Add(rootNode);

        AddDirectoryNodes(rootNode, rootDir);
        rootNode.Expand();
    }
    finally
    {
        treeViewMain.EndUpdate();
```

Рис. 6.1.2 – Метод LoadDirectoryTree

```
private void AddDirectoryNodes(TreeNode parentNode, DirectoryInfo dir)
{
    try
    {
        // спочатку всі підкаталоги
        DirectoryInfo[] subDirs;
        try
        {
            subDirs = dir.GetDirectories();
        }
        catch (UnauthorizedAccessException)
        {
            return;
        }
        catch (DirectoryNotFoundException)
        {
            return;
        }
        catch (IOException)
        {
            return;
        }
        catch (Exception)
        {
            return;
        }

        foreach (DirectoryInfo sub in subDirs)
        {
            var node = new TreeNode(sub.Name)
            {
                Tag = new TreeNodeTag { IsDirectory = true, FullPath = sub.FullName }
            };
            parentNode.Nodes.Add(node);
            try
            {
                AddDirectoryNodes(node, sub); // рекурсивний вхід у вкладену папку
            }
            catch (UnauthorizedAccessException)
            {
                // одна гілка недоступна — решта дерева будується далі
            }
            catch (DirectoryNotFoundException)
            {
            }
            catch (IOException)
            {
            }
            catch (Exception)
            {
            }
        }

        // потім файли безпосередньо в цій папці (не всередині підпапок — ті вже в окрем
        FileInfo[] files;
        try
        {
            files = dir.GetFiles();
        }
        catch (UnauthorizedAccessException)
        {
            return;
        }
        catch (DirectoryNotFoundException)
        {
            return;
        }
        catch (IOException)
        {
            return;
        }
        catch (Exception)
        {
            return;
        }

        foreach (FileInfo file in files)
        {
            var fileNode = new TreeNode(file.Name)
            {
                Tag = new TreeNodeTag { IsDirectory = false, FullPath = file.FullName }
            };
            parentNode.Nodes.Add(fileNode);
        }
    }
    catch (UnauthorizedAccessException)
    {
    }
    catch (DirectoryNotFoundException)
    {
    }
    catch (IOException)
    {
    }
    catch (Exception)
    {
    }
}
```

Рис. 6.1.3 – Метод AddDirectoryNodes

6.2. Пошук файлів і виведення повної інформації про знайдені екземпляри

Для виконання другого завдання було реалізовано механізм пошуку файлів за іменем або маскою. Користувач вводить шаблон пошуку у спеціальне текстове поле, наприклад *.txt, *.pdf або точну назву файлу, після чого натискає кнопку пошуку або клавішу Enter. Програма починає рекурсивний обхід вибраної директорії та всіх її підкаталогів.

Основним методом, що запускає операцію пошуку, є SearchFiles. У ньому виконується перевірка наявності кореневої директорії, перевірка коректності введеної маски, очищення попередніх результатів та запуск рекурсивної функції пошуку. Якщо жодного файлу не знайдено, програма виводить відповідне повідомлення користувачу.

Рекурсивний обхід підкаталогів реалізовано в методі SearchFilesRecursive. Для поточної папки програма спочатку отримує список файлів, які відповідають шаблону, за допомогою Directory.EnumerateFiles(currentPath, pattern), а потім отримує список підкаталогів через Directory.EnumerateDirectories(currentPath)

і рекурсивно викликає той самий метод для кожної вкладеної директорії. Такий підхід дозволяє виконувати повний глибокий пошук без обмеження рівня вкладеності.

Для кожного знайденого файлу формується елемент списку ListView, у якому відображається назва файлу, його тип і розмір. Додатково для рядка зберігається повний шлях до файлу в структурі ListItemTag, що дозволяє потім відкрити файл або показати його властивості.

Окремим важливим елементом реалізації є метод ShowFileInfo, який використовується для відображення повної інформації про вибраний файл. У цьому методі, на основі об’єкта FileInfo, виводяться такі характеристики: ім’я файлу, повний шлях, розмір у байтах, дата створення, дата останньої зміни та атрибути. Саме ця частина забезпечує відповідність другому завданню методички, оскільки дозволяє не лише знайти файл, а й надати користувачу повні відомості про нього.

У програмі також реалізовано коректну обробку помилок. Якщо під час обходу директорій виникає виняток UnauthorizedAccessException, DirectoryNotFoundException або IOException, пошук не завершується аварійно, а проблемний каталог пропускається. Якщо введена некоректна маска пошуку, помилка обробляється окремо, і користувач отримує зрозуміле повідомлення.

Таким чином, у межах другого завдання було реалізовано рекурсивний пошук файлів із можливістю виведення повної інформації про кожен знайдений екземпляр.

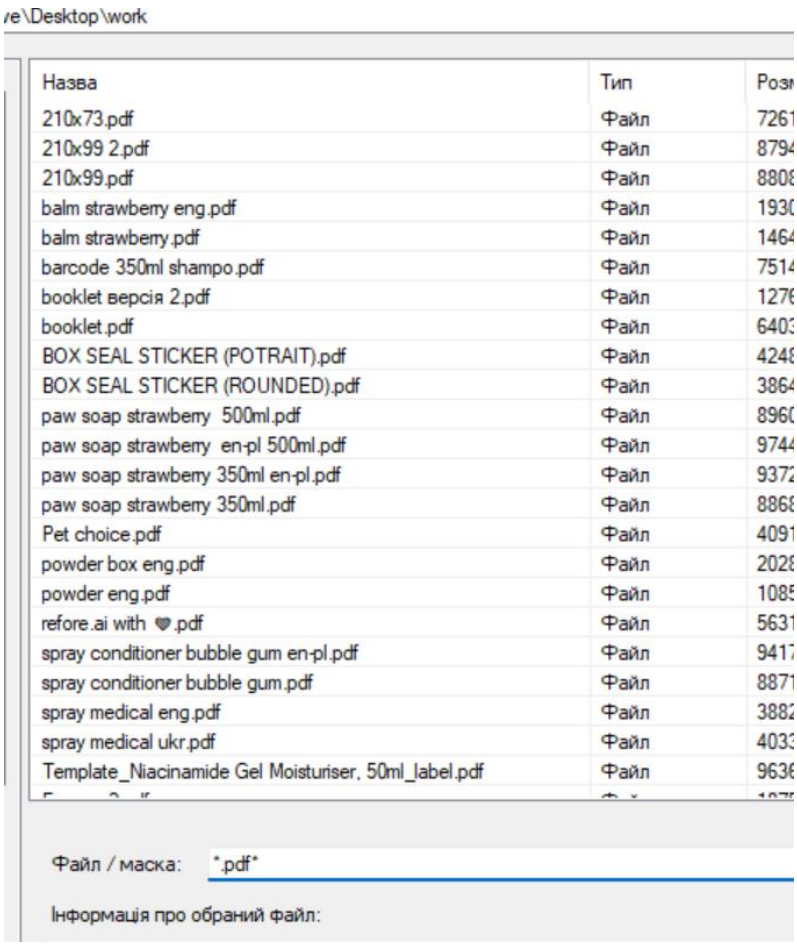

Рис. 6.2.1 – Введення маски файлу в поле пошуку та відображення результатів

```
/// пошук файлів за маскою з рекурсивним обходом дерева каталогів.
/// перевіряє корінь, далі делегує <see cref="SearchFilesRecursive"/> — помилка доступу в одній папці не зриває весь пошук.
private void SearchFiles(string rootPath, string pattern)
{
    listViewContents.Items.Clear();
    textBoxFileInfo.Clear();

    if (!Directory.Exists(rootPath))
    {
        MessageBox.Show(this, "Кореневий каталог не існує: " + rootPath, "Помилка", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    try
    {
        // рекурсія всередині ловить відмову доступу по кожній папці (без MessageBox на кожну)
        SearchFilesRecursive(rootPath, pattern);

        if (listViewContents.Items.Count == 0)
        {
            MessageBox.Show(this, "Файлів за заданою маскою не знайдено.", "Пошук", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
    catch (ArgumentException ex)
    {
        // некоректна маска для EnumerateFiles
        ShowError("Некоректний шаблон пошуку або шлях.", ex);
    }
    catch (Exception ex)
    {
        ShowError("Критична помилка під час запуску пошуку.", ex);
    }
}
```

Рис. 6.2.2 – Метод SearchFiles

```
private void SearchFilesRecursive(string currentPath, string pattern)
{
    //лише файли в цій папці
    try
    {
        foreach (string fullPath in Directory.EnumerateFiles(currentPath, pattern))
        {
            try
            {
                var fi = new FileInfo(fullPath);
                var item = new ListViewItem(fi.Name);
                item.SubItems.Add("Файл");
                item.SubItems.Add(fi.Length.ToString());
                item.Tag = new ListItemTag { IsDirectory = false, FullPath = fi.FullName };
                listViewContents.Items.Add(item);
            }
            catch (UnauthorizedAccessException)
            {
            }
            catch (IOException)
            {
            }
            catch (Exception)
            {
                // один файл міг зникнути або заблокуватися під час обходу
            }
        }
    }
    catch (UnauthorizedAccessException)
    {
    }
    catch (DirectoryNotFoundException)
    {
    }
    catch (IOException)
    {
    }
    catch (Exception)
    {
        // не вдалося прочитати вміст цієї папки — пропускаємо файли, але підпапки спробуємо окремо
    }

    // окремий try — навіть якщо перелік файлів у папці зірвався, підкаталоги іншої гілки можуть бути дос
    try
    {
        foreach (string subDir in Directory.EnumerateDirectories(currentPath))
        {
            SearchFilesRecursive(subDir, pattern);
        }
    }
```

Рис. 6.2.3 – Метод SearchFilesRecursive

```
private void ShowFileInfo(FileInfo file)
{
    var sb = new StringBuilder();
    sb.AppendLine("Ім'я файлу: " + file.Name);
    sb.AppendLine("Повний шлях: " + file.FullName);
    sb.AppendLine("Розмір: " + file.Length + " байт");
    sb.AppendLine("Дата створення: " + file.CreationTime.ToString("dd.MM.yyyy HH:mm:ss"));
    sb.AppendLine("Дата зміни: " + file.LastWriteTime.ToString("dd.MM.yyyy HH:mm:ss"));
    sb.AppendLine("Атрибути: " + file.Attributes);
    textBoxFileInfo.Text = sb.ToString();
}
```

Рис. 6.2.4 – Метод ShowFileInfo

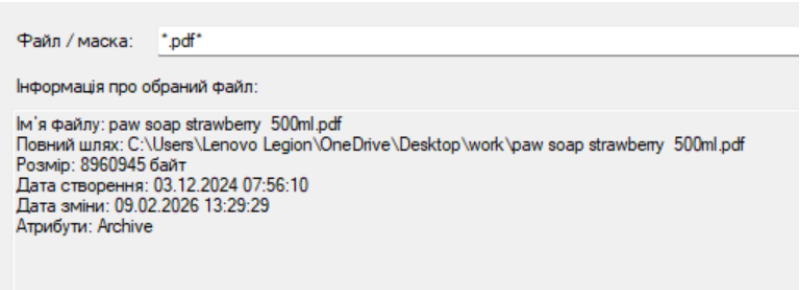

Рис. 6.2.4 – Відображення детальної інформації про вибраний файл у текстовому полі інтерфейсу

6.3. Реалізація графічного файлового менеджера

Третє завдання передбачало створення застосунку з графічним інтерфейсом, який реалізує основні функції файлового менеджера. Для виконання цього завдання було обрано технологію Windows Forms, яка дозволила створити зручне віконне середовище для роботи з директоріями та файлами.

Інтерфейс застосунку складається з кількох основних областей. У верхній частині розташовані текстове поле для введення або відображення шляху до кореневої папки, кнопка Огляд… для виклику FolderBrowserDialog та кнопка Завантажити, що запускає побудову дерева. Ліва частина вікна містить елемент TreeView, який показує повну структуру каталогів і файлів. Права верхня частина містить ListView, де відображається вміст вибраної папки або результати пошуку. У правій нижній частині розміщено поле для введення маски пошуку, кнопку пошуку та багаторядкове текстове поле для виведення детальної інформації про файл.

Для забезпечення взаємодії користувача з інтерфейсом були реалізовані обробники подій. Метод buttonBrowse_Click відповідає за відкриття діалогу вибору папки та підстановку обраного шляху в текстове поле. Метод buttonLoad_Click запускає побудову дерева каталогів через виклик LoadDirectoryTree. Метод buttonSearch_Click запускає пошук файлів за введеним шаблоном.

Для взаємодії з деревом і списком реалізовано обробники treeViewMain_AfterSelect та listViewContents_SelectedIndexChanged. Коли користувач вибирає папку в дереві, програма показує її вміст у списку за допомогою методу ShowDirectoryContents. Якщо користувач вибирає файл, програма перевіряє його існування за допомогою File.Exists, створює об’єкт FileInfo і викликає ShowFileInfo. Аналогічна логіка використовується і при виборі рядка в ListView.

Метод ShowDirectoryContents реалізує показ вмісту поточного каталогу в правій частині інтерфейсу. У ньому окремо додаються підкаталоги та файли, а для кожного елемента зберігається службова інформація через ListItemTag. Це дозволяє легко визначати тип обраного елемента та виконувати потрібну дію.

Додатковою можливістю файлового менеджера є відкриття файлу подвійним кліком. Цей функціонал реалізовано через обробник

listViewContents_DoubleClick, який викликає метод OpenSelectedFile. У цьому методі перевіряється, чи дійсно обраний елемент є файлом, чи існує він на диску, після чого використовується Process.Start з параметром UseShellExecute = true. Завдяки цьому файл відкривається у стандартній програмі Windows, яка асоційована з його форматом.

Окремо було реалізовано підтримку запуску пошуку клавішею Enter через обробник textBoxSearchPattern_KeyDown, а також єдиний механізм обробки помилок через методи ShowError і DescribeException, що забезпечує зручне інформування користувача про можливі проблеми під час роботи програми.

Отже, у межах третього завдання було створено повноцінний графічний файловий менеджер, який дозволяє переглядати структуру директорій, працювати зі списком файлів, виконувати пошук, переглядати властивості файлів і відкривати їх стандартними засобами операційної системи.

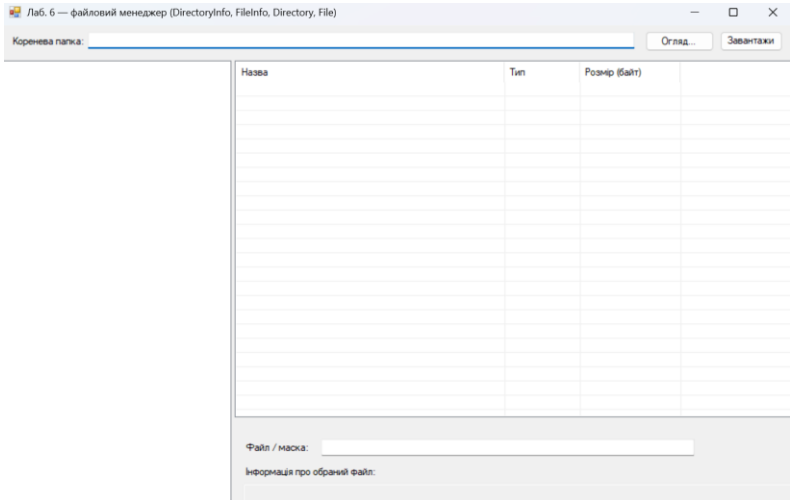

Рис. 6.3.1 – Загальний вигляд графічного інтерфейсу застосунку.

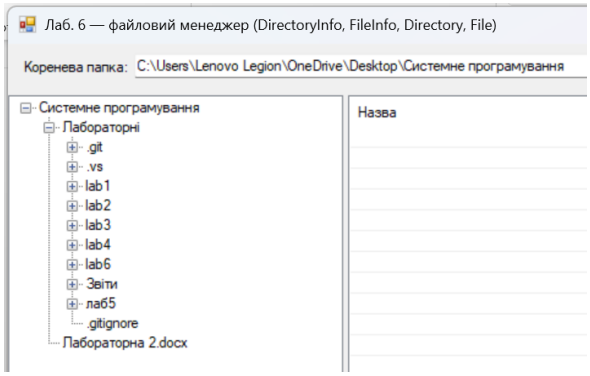

Рис. 6.3.2 – Вміст вибраної директорії у списку ListView.

```
/// Вміст однієї обраної папки у вигляді таблиці — підпапки та файли з розміром.
private void ShowDirectoryContents(string path)
{
    listViewContents.Items.Clear();

    try
    {
        if (!Directory.Exists(path))
        {
            MessageBox.Show(this, "Каталог не знайдено: " + path, "Помилка", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        var di = new DirectoryInfo(path);

        foreach (DirectoryInfo d in di.GetDirectories())
        {
            var item = new ListViewItem(d.Name);
            item.SubItems.Add("Папка");
            item.SubItems.Add("—");
            item.Tag = new ListItemTag { IsDirectory = true, FullPath = d.FullName };
            listViewContents.Items.Add(item);
        }

        foreach (FileInfo f in di.GetFiles())
        {
            var item = new ListViewItem(f.Name);
            item.SubItems.Add("Файл");
            item.SubItems.Add(f.Length.ToString());
            item.Tag = new ListItemTag { IsDirectory = false, FullPath = f.FullName };
            listViewContents.Items.Add(item);
        }
    }
    catch (UnauthorizedAccessException ex)
    {
        ShowError("Немає доступу до вмісту каталогу.", ex);
    }
    catch (DirectoryNotFoundException ex)
    {
        ShowError("Каталог не знайдено.", ex);
    }
    catch (IOException ex)
    {
        ShowError("Помилка читання каталогу.", ex);
    }
    catch (Exception ex)
    {
        ShowError("Не вдалося показати вміст каталогу.", ex);
    }
}
```

Рис. 6.3.3 – Фрагмент коду з файлу Form1.cs, який містить метод ShowDirectoryContents.

```
private void treeViewMain_AfterSelect(object sender, TreeViewEventArgs e)
{
    var tag = e.Node?.Tag as TreeNodeTag;
    if (tag == null)
    {
        return;
    }

    if (IsDirectoryNode(e.Node))
    {
        ShowDirectoryContents(tag.FullPath);
        textBoxFileInfo.Clear();
    }
    else if (IsFileNode(e.Node))
    {
        try
        {
            listViewContents.Items.Clear();
            // перевірка, що файл ще існує перед FileInfo
            if (!File.Exists(tag.FullPath))
            {
                MessageBox.Show(this, "Файл не знайдено на диску.", "Помилка", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }

            var fi = new FileInfo(tag.FullPath);
            ShowFileInfo(fi);
        }
        catch (Exception ex)
        {
            ShowError("Не вдалося прочитати інформацію про файл.", ex);
        }
    }
}
```

Рис. 6.3.4 – Фрагмент коду з файлу Form1.cs, який містить обробник treeViewMain_AfterSelect.

```
private void listViewContents_SelectedIndexChanged(object sender, EventArgs e)
{
    if (listViewContents.SelectedItems.Count == 0)
    {
        return;
    }

    var item = listViewContents.SelectedItems[0];
    var lit = item.Tag as ListItemTag;
    if (lit == null)
    {
        return;
    }

    try
    {
        if (lit.IsDirectory)
        {
            ShowDirectoryContents(lit.FullPath);
            textBoxFileInfo.Clear();
        }
        else
        {
            if (!File.Exists(lit.FullPath))
            {
                MessageBox.Show(this, "Файл не знайдено на диску.", "Помилка", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }

            var fi = new FileInfo(lit.FullPath);
            ShowFileInfo(fi);
        }
    }
    catch (Exception ex)
    {
        ShowError("Не вдалося обробити вибір у списку.", ex);
    }
}
```

Рис. 6.3.5 – Фрагмент коду з файлу Form1.cs, який містить обробник listViewContents_SelectedIndexChanged.

```
private void OpenSelectedFile()
{
    if (listViewContents.SelectedItems.Count == 0)
    {
        return;
    }

    var item = listViewContents.SelectedItems[0];
    var tag = item.Tag as ListItemTag;
    if (tag == null || tag.IsDirectory)
    {
        return;
    }

    if (!File.Exists(tag.FullPath))
    {
        MessageBox.Show(this, "Файл не знайдено на диску.", "Помилка", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    try
    {
        System.Diagnostics.Process.Start(new System.Diagnostics.ProcessStartInfo
        {
            FileName = tag.FullPath,
            UseShellExecute = true
        });
    }
    catch (Exception ex)
    {
        ShowError("Не вдалося відкрити файл у стандартній програмі.", ex);
    }
}
```

Рис. 6.3.6 – Фрагмент коду з файлу Form1.cs, який містить метод OpenSelectedFile.

```
private void textBoxSearchPattern_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        e.Handled = true;
        e.SuppressKeyPress = true;
        buttonSearch_Click(sender, EventArgs.Empty);
    }
}
```

Рис. 6.3.7 – Фрагмент коду з файлу Form1.cs, який містить обробник textBoxSearchPattern_KeyDown.

Висновок

У ході виконання лабораторної роботи було розроблено застосунок Lab6_FileIO_Manager, який реалізує основні можливості роботи з файловою системою у середовищі C#. У програмі виконано побудову повної ієрархічної структури директорій, реалізовано рекурсивний пошук файлів за маскою та відображення детальної інформації про знайдені файли. Також створено графічний інтерфейс файлового менеджера, що забезпечує зручну навігацію по каталогах, перегляд вмісту папок, перегляд властивостей файлів і відкриття файлів у стандартних програмах операційної системи. У процесі виконання роботи було закріплено практичні навички використання класів DirectoryInfo, FileInfo, Directory, File, а також опрацьовано принципи організації графічних застосунків Windows Forms для роботи з файловим введенням-виведенням.